

Sunday
8-Feb-2026

2nd JavaScript Class

Variable

Var - Deprecate

(isko use mat karo)

use let & Const

interview question

is let is hoisted?

- yes & explanation

~~because~~ It is ~~depo~~

Because it is temporal dead zone (TDZ)

Temporal Dead Zone (TDZ)

- A Temporal Dead Zone in JavaScript is the period from the start of a block scope (`{ }`) until a `let` or `const` variable is actually declared and initialized, during which the variable exists but is inaccessible, causing a `ReferenceError` if accessed. Unlike `var` variables, which are hoisted but `undefined`.

let variable

Example,

<pre>age = 65; console.log (age);</pre>					
<pre>let age = 24; console.log (age);</pre>	TDZ <table border="1"> <thead> <tr> <th>MP</th><th>CP</th></tr> </thead> <tbody> <tr> <td>age = undefined.</td><td></td></tr> </tbody> </table> → if you use let in TDZ then It's gonna through error	MP	CP	age = undefined.	
MP	CP				
age = undefined.					
<pre>age = 45; console.log (age);</pre>					

Const variable

you can't assign const variable to any one.
 you can only assign for one time.

Examples

```
Const age = 24;
```

- It has similar behaviour how let variable
- It also makes TDZ.

Functions

- function - a reusable set of instructions
- A block of code designed to perform a specific task.

Syntax

```
function functionName () {
    // code stats here
}
```

- add input called as parameters

ex,

```

function greetUser (x) {
  console.log ('Hey' + x);
}
  
```

function name Parameter

greetUser ('Piyush');

x value

(you can also add more than one parameter.)

- String interpolation ('')

you can add variable under the backtick

ex,

```

function greetUser (x, y, z) {
  console.log ('hey, `${x}` and `${y}`');
}
  
```

function names Parameters

greetUser ('Dipali', 'Sharma', '.');

interpolate the variable.

- Expressions

ex,

```

function add (num1, num2) {
  const result = num1 + num2;
  return result;
}
  
```

return is the last statement of the function

const r = add(2, 5)

for (let i = 0; i < 5; i++) {

console.log ('value of i is ' + i);
}

Output =

value of i is 0
↓
log +

value of i is 7

- Function inside function

ex,

```
function cartoon () {
  function cartoonInsideCartoon () {
    return 'Naruto';
  }
  return cartoonInsideCartoon;
}
```

function inside Function

return the function

Type of anime is function

```
const anime = cartoon ();
const x = anime ();
console.log (x);
```

called the function

assign the value

- you can call the function inside the function.
- Function can return any thing like String, number, or even function.

- alternate way to make function

ex,

```
let cartoon = function () {
  console.log ('Anime');
};
cartoon ();
```

ex,

```
age = 45;
console.log('Value of age is', age, 'Is Allowed?',
  isAllowedToVote(age));
```

```
var age = 24;
```

```
function isAllowedToVote (age) {
  return age >= 18;
}
```

mp	cp
age (variable)	undefined 45
function is AllowedToVote (age) {	
return age >= 18;	
}	

output:-

Value of age is 45 . Is Allowed? True

If define let & const insted of var
It gives you ReferenceError (TDZ)

```
var isAllowedToVote = function (age) {
  return age >= 18;
};
```

- undefined ka app called nahi kar sakte.

Arrow Function (=>)

- you can skip the function keyword, the return keyword and the curly brackets.

ex,

Function name
Parameters
return
 Const isAllowedToVote = (age) => age >= 18;

console.log(isAllowedToVote(23));

Const isUserAllowedToOpenBankAccount =
Too Parameters (age, minBalance) => {
 return age >= 18 && minBalance >= 5000;
 }
 console.log(isUserAllowedToOpenBankAccount(23, 6000));

- If you have more than one statement then you have to add paranthesis.
- there is no need to add paranthesis if you have one parameter
- If you use paranthesis then you have to use return keyword too.

Data Structure

- memory mai data ko ek perticular structure mei store karna

Data Structure of Javascript

- Array ([])

- An array is a single object used to store an ordered collection of multiple values under a single variable name.
- Arrays are dynamic, zero-indexed (starts with 0)
- you can use different type of data type like String, number, Boolean

ex,

```

const fruits = [
  'apple', 'cheeku', true, 'aada', 1, 2, 3, 4, 5,
  'santra', 'kela'
];

```

```

console.log(fruits[4]);

```

output

aada

```

console.log(fruits[10]);

```

Output

undefined

(no error for out of bound)

- you can also print using

```

console.log(fruits);

```

Output

apple

cheeku

true

aada

1

santra

2

kela

- add (end me kuch add)

```
fruits.push('Kiwi');
```

- Print the length

```
console.log(fruits.length);
```

- Check the value in array

```
console.log(fruits.includes('aadu'))
```

Output

true.

- Returns a copy of a section of an array.

```
const firstElement = fruits.slice(2, 5)
```

index 2 se 5 tak sare element
de do.

assignment

Queues and stack Implement Karne hai
using array

- `console.log({firstElement});`

- `fruits.unshift('1', '2', '3');`

- `console.log(fruits.indexOf());`

High Order Functions

- A function that takes another function as a parameter

ex,

```
function metaPyaraFunction (udharkFunction) {  
  return udharkFunction () + 40;  
}
```

call the function

Point towards cartoon

```
function cartoon () {  
  return 10;  
}
```

simple function

```
Console.log (metaPyaraFunction (cartoon));
```

cartoon is
Parameters

called the function

then $10 + 40$

Output

50

Ex,

```
function metaPyaraFunction (udharkFunction) {  
  return udharkFunction () + 40;  
}
```

call the function

$100 + 40$

```
function metaEkAurFunction () {  
  return 100;  
}
```

Parameter as function

```
console.log (metaPyaraFunction (metaEkAurFunction));
```

called the function

total = 140

forEach

ex,

```
const fruits = ['apple', 'cheeku', 'truu', 'badu', 'l', 'santra', 'Ø', 'kela'];
```

```
fruits.forEach((x, y) => console.log(`${x} ${y}`));
```

Build in forEach

calling function for every element.

String interpolation

```
function forEach (bataoKyakarnaHai) {
  for (let i = 0; i < fruits.length; i++) {
    bataoKyakarnaHai (fruits[i]);
  }
}
```

Map

makes new array

ex,

```
const nums = [1, 2, 3, 4, 5, 6]
const result = nums.map (e) => e * 2);
console.log (result);
```

output

[2, 4, 6, 8, 10, 12]

```
Const nums = [1, 2, 3, 4, 5, 6];
```

```
Const result = map(e) => e * 3);
```

call the Function

```
Console.log (result);
```

```
Function map (fn) {
```

```
  Const result = [];
```

```
  for (let i = 0; i < nums.length; i++) {
```

```
    Const currentElement = nums [i];
```

```
    Const num = fn (currentElement);
```

```
    result . push (num);
```

```
  }
```

```
  return result;
```

```
}
```

Output:-

```
[ 3, 6, 9, 12, 15, 18 ];
```

OR

```
Const nums = [1, 2, 3, 4, 5, 6];
```

```
Const result = nums . map (e) => e * 3);
```

```
Console.log (result);
```

```
Function map (fn) {
```

```
  Const result = [];
```

```
}
```